

An Exact String Matching Algorithm Based upon the Single Character Matching Mechanism

K. W. Liu and R. C. T. Lee
 Department of Computer Science and Information Engineering
 National Chi Nan University
 Puli, Nantou, Taiwan 545
 {s95321516,rctlee}@ncnu.edu.tw

Abstract

The exact string matching problem is to find all locations where a pattern P of length m matches a substring of a text T of length n . The single character matching rule is used by several researchers to solve this problem. In this paper, we propose a method to implement this single character rule. Our algorithm is easy to implement and quite efficient as shown by the experimental results. For instance, for a text string which is a DNA sequence with 36436 characters, when the length of the pattern is 35, the average number of pairs of characters compared is around 2000 while that needed for the Horspool algorithm is around 8000.

Keyword: Exact string matching, single character matching rule.

1. Introduction

In this paper, we are interested in the exact string matching problem which is defined as follows: We are given a text string $T = t_1 t_2 \cdots t_n$ and a pattern string $P = p_1 p_2 \cdots p_m$, we are asked to find all of the occurrences of P in T . Much research has been done in this field [1, 2, 4, 5, 6, 7, 8, 10, 11, 12, 13, 15, 16, 17, 21, 22, 23, 26, 27, 28, 30, 31, 32, 34, 37].

Most exact string matching algorithms use the slide window approach. That is, the pattern P along T , from left to right. If we find a mismatching, we will move the pattern. The key element in any such algorithm focuses on how to move the pattern. One approach is the single character matching scheme as described in the following figure.

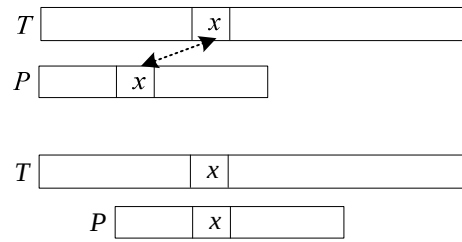


Fig 1: Illustration of single character matching

In the above figure, it is obvious that in order for the pattern P to match with a substring in T , we must slide the pattern in such a way that right most x located before location j in P must be aligned with the x in T . The question is “how do we select this particular x ?” In the next section, we shall review briefly three algorithms employing this approach.

2. A Review of Three Algorithms Employing the Single Character Matching Scheme

There are several algorithms which are based upon this rule [15, 23, 30]. For the Horspool algorithm presented in [15], the last character of the window is used as shown in Fig. 2.

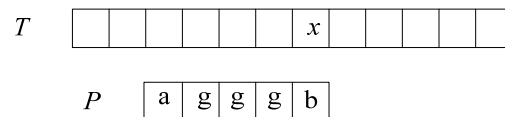


Fig 2: Illustration of the Horspool Algorithm

The Horspool algorithm starts by matching the last character of the window in T with the last character of P . If they match, we start to match characters to the right until a mismatch is found. Then we move P . The number of steps that we can move is determined by the

right most location of x in $P(1, m-1)$. If it is located in location d in P , we move $m-d+1$ steps. If x does not appear in P , we move m steps. Thus, for our case, if $x=a$, we can move P 4 steps to the right and if $x=g$, we can move P one step to the right and if $x=c$, we move 5 steps to the right.

The following shows how the Horspool algorithm works.

Let

$$T = \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ \hline a & g & a & g & a & a & g & c & t & a & g & g & g & a \\ \hline \end{array}$$

$$P = \begin{array}{|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 \\ \hline a & g & g & g & a \\ \hline \end{array}$$

Step 1

$$T = \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ \hline a & g & a & g & a & a & g & c & t & a & g & g & g & a \\ \hline \end{array}$$

*

$$P = \begin{array}{|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 \\ \hline a & g & g & g & a \\ \hline \end{array}$$

Step 2

$$T = \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ \hline a & g & a & g & a & a & g & c & t & a & g & g & g & a \\ \hline \end{array}$$

*

$$P = \begin{array}{|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 \\ \hline a & g & g & g & a \\ \hline \end{array}$$

Step 3

$$T = \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ \hline a & g & a & g & a & a & g & c & t & a & g & g & g & a \\ \hline \end{array}$$

*

$$P = \begin{array}{|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 \\ \hline a & g & g & g & a \\ \hline \end{array}$$

Here we found a match.

There are two other algorithms, namely Raita algorithm and Nebel algorithm, which use the single character rule. In both algorithms, the last character in the window is used. But they have different ways to match a character in T with a character in P . Horspool algorithm scans the pattern from right to left. In the Raita algorithm, we first compare the last character in pattern, then the first and finally the middle one. The Nebel algorithm scans the pattern from the least frequent character to the most frequent one.

3. Our Algorithm

Our algorithm is also based upon the single character matching rule. Consider the case

where we are using a window to match the pattern P . Suppose we select an i and the character matched with p_i is x . In P , the right most location of x before p_i is j . Then the number of steps which we can move P safely, if we have to do so, is $Shift(x, i)$ which is equal to $i - j + 1$. Let the alphabet set of P be A . Then the average number of steps which can be moved if location i is used is

$$MV(i) = \frac{\sum_{x \in A} Shift(x, i)}{|A|}.$$

We calculate $MV(i)$ for $1 \leq i \leq m$ and we choose the i with the largest $MV(i)$. Consider $P = aacatgggtattgatcttg$. For $i = 19$ we have $Shift(a, 19) = 5$, $Shift(g, 19) = 6$, $Shift(t, 19) = 1$, $Shift(c, 19) = 3$. Thus,

$$MV(19) = \frac{(5+6+1+3)}{4} = 3.75. \quad \text{It can also be seen that } MV(16) = \frac{19}{4} = 4.75.$$

To find the largest $MV(i)$, we perform a linear scan of P .

Let $A = a_1, a_2, \dots, a_c$ be the set of alphabets. We

define $d_i(a_j)$ as follows: If a_j appears

before p_{i+1} , $d_i(a_j)$ is the right most location of a_j before location $(i+1)$; otherwise,

$d_i(a_j) = 0$. Thus, for every i , we have a

$V_i = (d_i(a_1), d_i(a_2), \dots, d_i(a_c))$. However, when

we compute V_{i+1} , we only have to change one

element of V_i . That is, we only change

$d_{i+1}(p_{i+1})$ to be $i+1$. Once we get the V_i

vector, we know that the nearest a_j to the left

of location $i+1$ has $i+1 - d_i(a_j)$ positions.

For each i , we compute

$$MV(i) = \sum_{j=1}^c i+1 - d_i(a_j).$$

We only remember the V_i with the largest $MV(i)$. Suppose the best i is found, let $i_b = i+1$. Let $Shift(x)$, with one parameter, be a vector who stores the distance between the character x and p_{i_b} , where $x \in A$, x is right most and before p_{i_b} . We then calculate $Shift(x)$ as follows:

$$Shift(x) = i_b + 1 - d_{i_b}(x), \quad x \in A.$$

In the searching phase, in each search window, we first compare p_{i_b} with the corresponding

character y of the window of T . If p_{i_b} is equal to y , we try to compare the rest of characters of the window with the corresponding characters of P . If p_{i_b} does not match with y or there is any mismatch in the rest of comparisons, we move P $Shift(y)$ positions to the right.

Our preprocessing algorithm for determining i_b and shift table is given below.

Input $P = p_1p_2\dots p_m$

Output i_b and shift table

```

1   $A = \{a_1, a_2, \dots, a_c\}$  /*the set of alphabets*/
2  for  $i = 1 \dots c$ 
3       $d(a_i) = 0$ 
4  end for
5   $Max = 0$ 
6   $Total = c$  /*alphabet size */
7  for  $i = 1 \dots |P|-1$ 
8       $d(p_i) = i$ 
9       $Total = 0$ 
10     for  $j = 1 \dots c$ 
11          $Total = Total + i - d(a_j) + 1$ 
12     end for
13     if ( $Total > Max$ )
14          $i_b = i + 1$ 
15         for  $x = 1 \dots c$ 
16              $Shift(a_x) = i + 1 - d(a_x)$ 
17         end for
18          $Max = Total$ 
19     end if
20 end for
```

Preprocessing Algorithm.

Our searching algorithm is given below.

Input $T = t_1t_2t_3\dots t_n$

$P = p_1p_2p_3\dots p_m$

Output the position of T where P appears

```

1  /* Searching */
2   $j = 1$ 
3  while ( $j \leq n - m + 1$ )
4       $y = t_{j+i_b}$ 
5      if ( $p_{i_b} == y$ )
6           $q = m$ 
7          while ( $q > 0$  &  $p_q == t_{j+q}$ )  $q = q - 1$ 
8          if ( $q == 0$ )
9              print  $j$ 
10         end if
11     end if
12      $j = j + Shift(y)$ 
13 end while
```

Searching Algorithm

We now give an example.

We are given $T = \text{agtgtcagagctaca}$ and $P = \text{agagcta}$.

Preprocessing Phase:

$d(a) = 0, d(g) = 0, d(t) = 0, d(c) = 0, Max = 0$

When $i = 1$, $d(a) = 1$ and $Total = 7$. The value $Total$ is greater than Max , so we reset $Shift$ table. $Shift(a) = 1$, $Shift(g) = 2$, $Shift(t) = 2$, $Shift(c) = 2$, now $Max = 7$

When $i = 2$, $d(g) = 2$ and $Total = 9$. So we reset $Shift$ table. $Shift(a) = 2$, $Shift(g) = 1$, $Shift(t) = 3$, $Shift(c) = 3$, $Max = 9$

When $i = 3$, $d(a) = 3$ and $Total = 11$. $Shift(a) = 1$, $Shift(g) = 2$, $Shift(t) = 4$, $Shift(c) = 4$, $Max = 11$

When $i = 4$, $d(g) = 4$ and $Total = 13$. $Shift(a) = 2$, $Shift(g) = 1$, $Shift(t) = 5$, $Shift(c) = 5$, $Max = 13$

When $i = 5$, $d(c) = 4$ and $Total = 12$. The value $Total$ is now less than Max , we don't need to reset $Shift$ table.

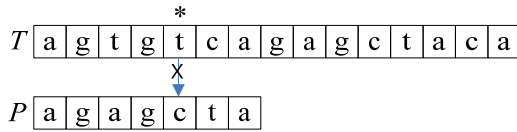
When $i = 6$, $d(t) = 4$ and $Total = 10$. The value $Total$ is now less than Max , we don't need to reset $Shift$ table.

After the preprocessing stage, we obtain the following: $i_b = 5$. The corresponding shift table is:

a	g	t	c
2	1	5	5

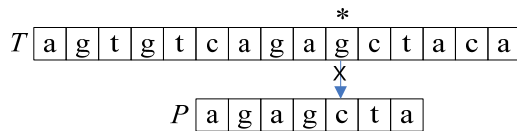
The searching is done as follows:

Step 1



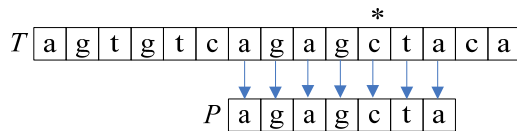
$Shift(t)$ is 5. We move P 5 positions.

Step 2



$Shift(g)$ is 1. We move P 1 positions.

Step 3



Here we found a match. $Shift(c)$ is 5. We move P 5 positions. Out of range, searching phase is terminated.

4. Experimental Results

We performed experiments on a DNA sequence with size 36436 and used it as the text string T . We repeatedly randomly took a substring from a DNA and used it as the pattern string P . We used six different lengths of P . For each length, we randomly extracted a substring with such length twenty times. That is, for each length, twenty P 's were generated. The Horspool algorithm and our algorithm were tested on all of the data. We recorded the average number of character comparisons in the searching phase for both algorithms and the result is shown in the following chart. It can be seen that our algorithm needs much smaller number of character comparisons than that need by the Horspool algorithm.

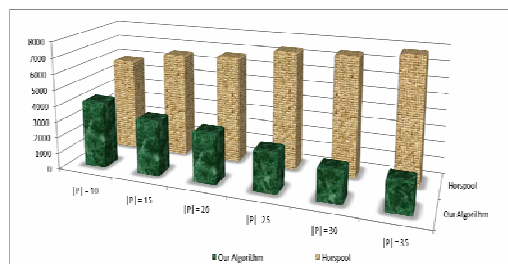


Fig 3: The average number of character comparisons in searching phase for Horspool and our algorithm, text length 36436, pattern length 10, 15, 20, 25, 30, 25.

5. Concluding Remarks and Future Research

We have proposed a rule to select an appropriate character so that the single character approach can become efficient. Experimental results show that our algorithm performs better than the Horspool algorithm. We believe that our idea can be used to solve approximate string matching problems [3, 14, 18, 19, 20, 24, 25, 29, 33, 35, 36].

References

- [1] A. V. Aho. Algorithms for finding patterns in strings. *Handbook of Theoretical Computer Science, Volume A, Algorithms and complexity*, 1990, pp. 255-300.
- [2] A. Apostolico and M. Crochemore. Optimal canonization of all substrings of a string. *Information and Computation*, Vol. 95, No. 1, 1991, pp. 76-95.
- [3] A. Amir, D. Keselman, G. M. Landau, M. Lewenstein, N. Lewenstein and M. Rodeh. Text indexing and dictionary matching with one error. *Journal of Algorithms*, Vol. 37, 2000, pp. 309-325.
- [4] R. A. Baeza-yates and M. Régnier. Average running time of the Boyer-Moore-Horspool algorithm. *Theoretical Computer Science*, Vol. 92, No. 1, 1992, pp. 19-31.
- [5] D. Beauquier, J. Berstel and P. Chrétienne. *Éléments d'algorithmique*. 1992, Chapter 10, pp. 337-377, Masson, Paris.
- [6] R. A. Baeza-yates and G. H. Gonnet. A new approach to text searching. *Communication of the ACM*, Vol. 35, No. 10, 1992, pp. 74-82.
- [7] R. S. Boyer and J. S. Moore. A fast string searching algorithm. *Communications of the ACM*, Vol. 20, 1977, pp. 762-772.
- [8] T. Berry and S. Ravindran. A fast string matching algorithm and experimental results. *Proceedings of the Prague Stringology Club Workshop '99*, 1999, pp. 16-26.

- [9] L. Colussi. Correctness and efficiency of the pattern matching algorithms. *Information and Computation*, Vol. 95, No. 2, 1991, pp. 225-251.
- [10] M. Crochemore, A. Czumaj, L. Gasieniec, S. Jarominek, T. Lecroq, W. Plandowski and W. Rytter. Deux méthodes pour accélérer l'algorithme de Boyer-Moore. *Théorie des Automates et Applications, Actes des 2^e Journées Franco-Belges*, D. Krob ed., Rouen, France, 1992, pp. 45-63.
- [11] M. Crochemore and C. Hancart. Pattern matching in strings. *Algorithms and Theory of Computation Handbook*, 1999, Chapter 11, pp. 11-1--11-28, CRC Press Inc., Boca Raton, FL.
- [12] T. H. Corment, C. E. Leiserson and R. L. Rivest. *Introduction to algorithms*, 1990, Chapter 34, pp. 853-885.
- [13] M. Crochemore and D. Perrin. Two-way string-matching. *Journal of the ACM*, Vol.38, No. 3, 1991, pp. 651-675.
- [14] K. Fredriksoon and G. Navarro. Average-optimal single and multiple approximate string matching. *ACM Journal of Experimental Algorithmics*, Vol. 9, Article No. 1.4, 2004, pp. 1-47.
- [15] R. N. Horspool. Practical fast searching in strings. *Software - Practice & Experience*, Vol. 10, No. 6, 1980, pp. 501-506.
- [16] C. Hancart. Une analyse en moyenne de l'algorithme de Morris et Pratt et de ses raffinements. *Théorie des Automates et Applications*, 1992.
- [17] C. Hancart. Analyse exacte et en moyenne d'algorithmes de recherche d'un motif dans un texte, Ph. D. Thesis. *University Paris 7, France*, 1993.
- [18] H. Hyvärö. Bit-parallel approximate string matching algorithms with transposition. *Journal of Discrete Algorithms*, Vol. 3, 2005, pp. 215-229.
- [19] N. D. Huynh, W. K. Hon, T. W. Lam and W. K. Sung. Approximate string matching using compressed suffix arrays. *Theoretical Computer Science*, Vol. 352, 2006, pp. 240-249.
- [20] H. Hyvärö and G. Navarro. Bit-parallel witnesses and their applications to approximate string matching. *Algorithmica*, Vol 4, No. 3, 2005, pp. 203-231.
- [21] D. E. Knuth, J. H. Morris and V. R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, Vol. 6, No. 1, 1977, pp. 323-350.
- [22] R. M. Karp and M. O. Rabin. Efficient randomized pattern matching algorithms. *IBM Journal of Research and Development Archive*, Vol. 31, No. 2, 1987, pp. 249-260.
- [23] M. E. Nebel. Fast string matching by using probabilities: On an optimal mismatch variant of Horspool's algorithm. *Theoretical Computer Science*, Vol. 359, 2006, pp. 329-343.
- [24] G. Navarro and R. Baeza-Yates. Very fast and simple approximate string matching. *Information Processing Letters*, Vol. 72, 1999, pp. 65-70.
- [25] G. Navarro and R. Baeza-Yates. A hybrid indexing method for approximate string matching. *Journal of Discrete Algorithms*, Vol. 1, No.1, 2000, pp. 205-239.
- [26] J. H. Morris and V. R. Pratt. A linear pattern matching algorithm. *Technical Report 40*, 1970, University of California, Berkeley.
- [27] T. Lecroq. A variation on the Boyer-Moore algorithm. *Theoretical Computer Science*, Vol. 92, No. 1, 1992, pp. 119-144.
- [28] T. Lecroq. Experimental results on string matching algorithms. *Software - Practice & Experience*, Vol. 25, No. 7, 1995, pp. 727-765.
- [29] G. Landau and U. Vishkin. Fast parallel and serial approximate string matching. *Journal of algorithms*, Vol. 10, 1989, pp. 157-169.
- [30] T. Raita. Tuning the Boyer - Moore - Horspool string searching algorithm. *Software-Practice & Experience*, Vol. 22, No. 10, 1992, pp. 879-884.
- [31] D. M. Sunday. A very fast substring search algorithm. *Communications of the ACM*, Vol. 33, No. 8, 1990, pp. 32-142.
- [32] P. D. Smith. Experiments with a very fast substring search algorithm. *Software - Practice & Experience*, Vol. 21, No. 10, 1991, pp. 1065-1074.
- [33] I. Simon. String matching algorithms and automata. *Proceedings of 1st American Workshop on String Processing*, 1993.
- [34] G. A. Stephen. String searching algorithms. *World Scientific*, 1994.
- [35] J. Tarhio and E. Ukkonen. Approximate Boyer-Moore string matching. *SIAM Journal Computing*, Vol. 22, 1993, pp. 243-260.

- [36] W. Sun and U. Manber. Fast text searching: allowing errors. *Communications of the ACM*, Vol. 35, 1992, pp. 83-91.
- [37] R. F. Zhu and T. Takaoka. On improving the average case of the Boyer-Moore string matching algorithm. *Journal of Information Processing*, Vol. 10, No. 3, 1987, pp. 173-177.